



**Tablero de Indicadores de Finanzas Personales en NipinFinan: Integración de
JavaScript y Diseño Navegacional OOHDM**

Luciano Fohrholtz

Facultad de Ingeniería, Tecnología y Arquitectura, Universidad de la Cuenca del Plata

Paradigmas y Lenguajes de Programación III, Comisión "A"

Ing. Sicardi Dante

30 de septiembre de 2026



Tablero de Indicadores de Finanzas Personales en NipinFinan: Integración de JavaScript y Diseño Navegacional OOHDM

NipinFinan es un sistema web de gestión y análisis de economía personal que permite registrar ingresos y gastos, organizarlos por categoría y consultar el dinero disponible. En la primera etapa del proyecto se construyeron las pantallas con HTML y CSS. En esta actividad evaluativa se incorporó JavaScript al resumen del inicio para convertirlo en un tablero calculado a partir de los movimientos disponibles.

El objetivo de este informe es describir la implementación individual de la Opción 2, Tablero de Indicadores / Panel de Métricas, y fundamentar su organización mediante OOHDM. Se presentan los eventos, la carga asíncrona, los cálculos, la actualización del DOM y las verificaciones realizadas.

El desarrollo corresponde a Luciano Fohrholtz y complementa el calculador de tarifas de Santiago Gonzalez, correspondiente a la Opción 5. Ambos módulos funcionan en un único app.js. El trabajo se encuentra en la rama *ae2-gonzalez_santiago-fohrholtz_luciano* del repositorio NipinFinan (Fohrholtz y Gonzalez, 2026).

Consigna y Tarjeta Elegida

La consigna transversal exige capturar eventos mediante `addEventListener()` en `app.js`, sin atributos `onclick` ni `onsubmit` en el HTML, y cargar datos mediante `fetch()` con `async/await` desde un archivo JSON local. La Opción 2 agrega controles para consultar métricas, calcular totales y modificar el DOM, junto con una fundamentación OOHDM que distinga el dominio del nodo navegacional.



Tabla 1

Correspondencia entre la consigna y la implementación

Requisito	Implementación
Eventos desacoplados	Selectores y botones registrados con <code>addEventListener()</code> en <code>js/app.js</code> .
HTTP y asincronía	<code>cargarEjemplo()</code> utiliza <code>fetch()</code> , <code>await</code> y <code>response.json()</code> .
Datos locales JSON	<code>data/metricas.json</code> contiene movimientos ficticios de demostración.
Cálculo de métricas	Totales, saldo, porcentaje, cantidad y categoría de mayor gasto.
Actualización del DOM	Tarjetas, barras, estados y comparación mensual sin recargar.
Fundamentación U1	Movimiento como dominio y Resumen como nodo navegacional OOHDM.

Desarrollo

Módulo Seleccionado y Archivos

Se mejoró la sección Resumen del inicio, identificada como `#resumen` en `index.html`. La interfaz conserva el estilo del sitio: tipografía Arial, fondo claro, tonos azules y estados verdes o rojos. Los estilos particulares del tablero se encuentran en `css/resumen.css` para facilitar su mantenimiento.



Tabla 2

Archivos utilizados para el tablero

Archivo	Responsabilidad
index.html	Estructura de tarjetas, controles, mensajes y distribución de gastos.
js/app.js	Eventos, lectura de datos, petición HTTP, cálculos y renderizado. Compartido con el registro.
css/resumen.css	Estilos del resumen y adaptación a pantallas pequeñas.
data/metricas.json	Conjunto de movimientos de ejemplo para demostrar el tablero.
README.md	Descripción del aporte individual y ejecución, agregada debajo del trabajo de Santiago.

Desacoplamiento de Eventos

Los eventos se registran desde JavaScript y no mediante atributos HTML. Los selectores escuchan change; los botones escuchan click. Esto permite cambiar el comportamiento sin mezclarlo con el marcado de la página (MDN Web Docs, s. f.-a). También se atienden eventos pageshow y storage para actualizar la vista cuando corresponde.

Figura 1

Registro de los controles mediante addEventListener()

```
mes.addEventListener("change", mostrarResumen);
categoria.addEventListener("change", mostrarResumen);
botonActualizar.addEventListener(
  "click", () => actualizarDatos()
);
botonDemo.addEventListener(
  "click", () => actualizarDatos(
    modo === "real" ? "demo" : "real", true
  )
);
```

Nota. Fragmento de js/app.js; se ajustaron los saltos de línea para facilitar la lectura.



Fuentes de Datos

El modo inicial, Mis movimientos, lee la clave `nipinfinan_movimientos` de `localStorage`. Son las operaciones que el usuario registró en ese navegador y origen; el tablero solo las consulta. Este almacenamiento es local al navegador y no constituye una cuenta sincronizada ni una base de datos de usuarios (MDN Web Docs, s. f.-c).

El botón Ver demostración activa una segunda fuente, `data/metricas.json`. La pantalla identifica este modo como Datos de ejemplo y permite volver a Mis movimientos. Los ejemplos no se guardan en `localStorage` ni se mezclan con los datos reales. Así se puede probar la pantalla incluso cuando todavía no hay movimientos propios.

Carga Asíncrona con Fetch

La función `cargarEjemplo()` solicita el archivo JSON por HTTP con `fetch()` y espera su respuesta mediante `await`. Se verifica `response.ok` antes de convertir la respuesta con `response.json()`, ya que una respuesta HTTP de error debe tratarse explícitamente (MDN Web Docs, s. f.-b). Al actualizar la demostración se realiza una nueva petición.

Figura 2

Carga del archivo de movimientos de ejemplo

```
async function cargarEjemplo() {
  const respuesta = await fetch(
    "data/metricas.json", { cache: "no-store" }
  );
  if (!respuesta.ok) {
    throw new Error("HTTP " + respuesta.status);
  }
  const datos = await respuesta.json();
  return datos.movimientos;
}
```

Nota. Fragmento de `js/app.js`; se ajustaron los saltos de línea para facilitar la lectura.

El archivo contiene 22 movimientos distribuidos entre julio, agosto y septiembre de 2026. El modo de demostración selecciona inicialmente el mes más reciente del JSON; el



modo real utiliza el mes local actual. La carga con fetch satisface el requisito de asincronía; los movimientos propios continúan utilizando el almacenamiento del registro existente.

Indicadores y Cálculos

Tabla 3

Indicadores del resumen

Indicador	Cálculo o criterio
Ingresos	Suma de total en movimientos de tipo ingreso.
Gastos	Suma de total en movimientos de tipo gasto.
Saldo	Ingresos menos gastos.
Porcentaje de saldo	Saldo / ingresos $\times$ 100. Se muestra un guion si no hay ingresos.
Cantidad	Número de movimientos del contexto seleccionado.
Categoría de mayor gasto	Agrupación de gastos por categoría y búsqueda del importe máximo; se contemplan empates.

Se utiliza el campo total que ya contiene los descuentos o recargos aplicados por el calculador. No se vuelve a calcular una tarifa ni se multiplica el importe por la cantidad de cuotas. Para reducir errores de redondeo, los importes se suman en centavos enteros y se convierten a moneda al mostrarlos.

Figura 3

Suma de importes finales en centavos

```
function sumar(lista, tipo) {  
  return lista  
    .filter(m => m.tipo === tipo)  
    .reduce(  
      (total, m) =>  
        total + Math.round(m.total * 100),  
      0  
    );  
}
```

Nota. Fragmento de js/app.js; se ajustaron los saltos de línea para facilitar la lectura.



El porcentaje conserva el signo del saldo: si los gastos superan los ingresos, el resultado es negativo. Por ejemplo, ingresos de \$100,10 y gastos de \$120,25 generan un saldo de -\$20,15 y un porcentaje aproximado de -20,1 %. La ausencia de ingresos se distingue de un porcentaje igual a cero.

Filtros y Distribución de Gastos

El selector de período permite consultar un mes y año o Todo el período; el selector de categoría incluye Todas. Se filtran los movimientos en memoria y se recalculan los indicadores sobre la misma selección. El botón Actualizar resumen vuelve a consultar la fuente activa.

Los gastos se agrupan por categoría y se muestran mediante barras horizontales ordenadas de mayor a menor importe. Cada barra presenta el importe y su participación en los gastos seleccionados. Se utilizan elementos HTML y CSS, sin bibliotecas de gráficos. Si varias categorías comparten el máximo, se muestran los nombres empatados.

Comparación Mensual

Cuando se selecciona un mes, el tablero compara ingresos y gastos con el mes calendario anterior, conservando el filtro de categoría. La comparación de enero utiliza diciembre del año previo. Si no existen registros anteriores se informa esa situación; al seleccionar Todo el período, la comparación se oculta.

Se muestran diferencias absolutas: ingresos actuales menos ingresos anteriores y gastos actuales menos gastos anteriores. Un aumento de gastos se interpreta como mayor gasto, y no como una mejora automática de la economía personal.



Actualización del DOM y Manejo de Errores

Las funciones `mostrarResumen()` y `mostrarBarras()` actualizan textos, tarjetas, barras y mensajes sin recargar la página. Los valores de datos se incorporan con `textContent`. Durante una carga se deshabilitan los botones involucrados para evitar solicitudes repetidas.

La implementación contempla fallas de red, respuestas HTTP incorrectas, almacenamiento ilegible y movimientos inválidos. Los registros inválidos se omiten con una advertencia; una carga fallida conserva la última vista correcta y permite reintentar. El estado vacío informa que no hay movimientos y ofrece acceso al registro.

Integración en un Único App.js

El archivo compartido contiene dos bloques de inicialización. El registro se activa cuando existe `#form-movimiento` y el resumen cuando existe `#resumen`. Esta comprobación permite usar el mismo script en ambas páginas sin intentar acceder a controles inexistentes.

La integración conserva la lógica de tarifas, cuotas, cupones y guardado del módulo de Santiago. El tablero consume su importe final, por lo que las operaciones registradas pueden consultarse en el listado y en el resumen del inicio. Los archivos específicos de tarifas y el informe del compañero no se modificaron para elaborar este informe.

Fundamentación U1: Diseño Navegacional OOHDM

OOHDM distingue el modelo conceptual, la organización navegacional y la interfaz. Los nodos presentan vistas de la información del dominio, y los contextos organizan su exploración según la tarea del usuario (Rossi, Schwabe y Lyardet, s. f.). Esta separación se aplica al tablero de la siguiente manera.



Tabla 4

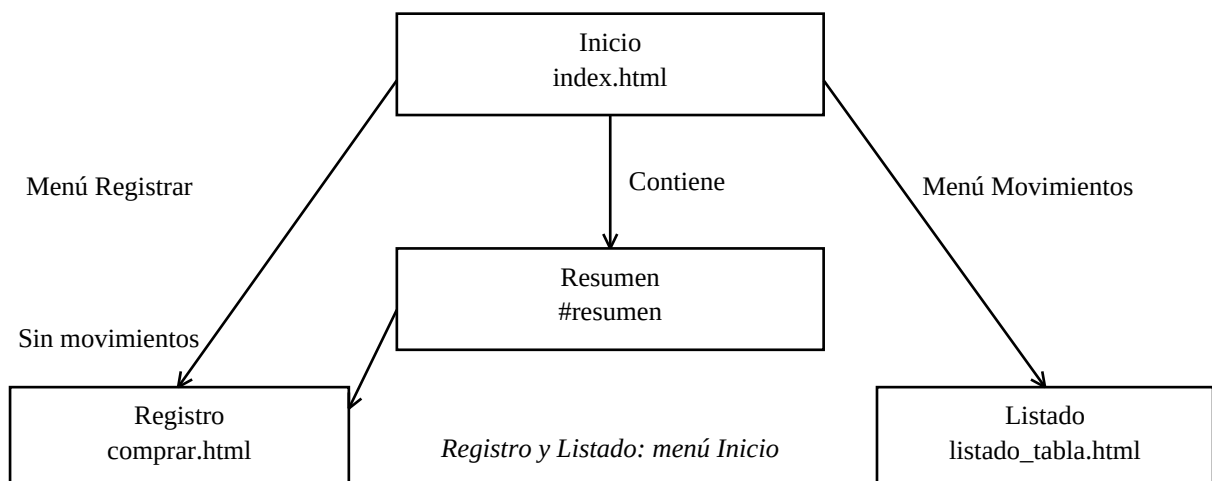
Aplicación de OOHDM a NipinFinan

Nivel	Elemento	Aplicación
Dominio	Movimiento	Operación con tipo, fecha, categoría e importe final.
Navegación	Resumen	Vista derivada de movimientos que presenta totales y distribución.
Contexto	Período y categoría	Selección de los movimientos que se exploran sin modificar las operaciones.
Interfaz	Tarjetas y controles	Presentación del resumen mediante selectores, indicadores y barras.

Movimiento es una clase conceptual del dominio; esta fundamentación no requiere implementar una clase JavaScript. Resumen es un nodo navegacional ubicado dentro del inicio, no una nueva entidad almacenada. Una tarjeta visual es parte de la interfaz del nodo, mientras que los selectores cambian su contexto.

Figura 4

Mapa navegacional del inicio y del resumen



Nota. Contiene indica ubicación dentro de la página. Las demás relaciones corresponden a enlaces o menús; Registro y Listado permiten regresar al Inicio. Elaboración propia.



El usuario registra una operación, vuelve al inicio para consultar su resumen y puede acceder al listado para revisar el detalle. La navegación conecta las pantallas existentes, mientras que los filtros reorganizan la información dentro del nodo Resumen. La etiqueta de la fuente diferencia la consulta real de la demostración.

Pruebas Realizadas

Durante la implementación se ejecutaron diez grupos de verificaciones en Chromium mediante Playwright, con el sitio servido por HTTP y datos de prueba en un navegador aislado. Las verificaciones se repitieron después de unificar los módulos en app.js. Se trata de grupos de comprobaciones, no de un conteo de aserciones individuales.

Tabla 5

Grupos de verificación y resultados

Grupo	Comprobación	Resultado
1	Estado vacío, ceros y porcentaje sin ingresos.	Correcto
2	Decimales, saldo negativo y registros inválidos.	Correcto
3	Total final, filtros, barras y comparación.	Correcto
4	Petición HTTP real, actualización y separación de fuentes.	Correcto
5	Revisión visual y ancho móvil de 375 px sin desbordamiento.	Correcto
6	JSON no disponible, reintento y recuperación.	Correcto
7	Almacenamiento ilegible y recuperación.	Correcto
8	Empate de categorías con mayor gasto.	Correcto
9	Comparación enero/diciembre del año anterior.	Correcto
10	Registro con crédito, listado y cupón UCP10.	Correcto



No se detectaron errores JavaScript sin capturar durante estas verificaciones. La

Tabla 6 resume los valores de la demostración sin filtro de categoría; permite contrastar los totales calculados con los datos del JSON.

Tabla 6

Totales mensuales del archivo de demostración

Mes 2026	Ingresos	Gastos	Saldo	Saldo %	Cant.
Julio	\$150.000,00	\$80.500,00	\$69.500,00	46,3 %	6
Agosto	\$175.000,00	\$97.000,00	\$78.000,00	44,6 %	7
Septiembre	\$185.000,00	\$115.700,75	\$69.299,25	37,5 %	9

En septiembre, Alimentación es la categoría de mayor gasto, con \$35.000,00.

Respecto de agosto, los ingresos aumentan \$10.000,00 y los gastos \$18.700,75. En la prueba de integración, un gasto de \$10.000,00 con crédito en tres cuotas y recargo del 8 % genera un total de \$10.800,00: ese es el importe que usa el resumen, sin volver a multiplicarlo por las cuotas.

Instrucciones de Ejecución

Se debe abrir la rama ae2-gonzalez_santiago-fohrholtz_luciano en VS Code y ejecutar Live Server desde index.html. Como alternativa, desde la carpeta del sitio se puede ejecutar `python -m http.server 8000` y acceder a `http://localhost:8000/index.html`.

Para comprobar el flujo real, registrar un ingreso o gasto, regresar al inicio y cambiar el período o la categoría. Luego seleccionar Todo el período y Actualizar resumen. Para revisar la carga HTTP, presionar Ver demostración, explorar las barras y la comparación, y volver a Mis movimientos para comprobar que los ejemplos no se guardaron.



La demostración necesita un servidor HTTP para cargar el JSON; no se debe abrir el HTML con doble clic. Debe mantenerse el mismo protocolo, host y puerto para acceder al mismo localStorage. El archivo de demostración es una fuente de lectura y no recibe los movimientos que se guardan desde el formulario.

Conclusión

La actualización convierte el resumen del inicio en un tablero de finanzas personales con métricas calculadas, filtros, distribución por categoría y comparación mensual. Los eventos están desacoplados del HTML y la demostración utiliza fetch() con async/await. Los movimientos propios y los de ejemplo se mantienen separados.

La organización se fundamenta mediante OOHDM al distinguir Movimiento como dominio, Resumen como nodo y período/categoría como contexto. La integración en un único app.js permite trabajar junto al calculador existente y aprovechar sus importes finales sin duplicar descuentos, recargos ni cuotas.

Referencias

- Fohrholtz, L., y Gonzalez, S. (2026). *NipinFinan* [Código fuente, rama ae2-gonzalez_santiago-fohrholtz_luciano]. GitHub.
https://github.com/LuchoFohrholtz/NipinFinan/tree/ae2-gonzalez_santiago-fohrholtz_luciano
- MDN Web Docs. (s. f.-a). *EventTarget: addEventListener()*.
<https://developer.mozilla.org/es/docs/Web/API/EventTarget/addEventListener>
- MDN Web Docs. (s. f.-b). *Uso de Fetch*. https://developer.mozilla.org/es/docs/Web/API/Fetch_API/Using_Fetch
- MDN Web Docs. (s. f.-c). *Window: localStorage*.
<https://developer.mozilla.org/es/docs/Web/API/Window/localStorage>
- Rossi, G., Schwabe, D., y Lyardet, F. (s. f.). *Web application models are more than conceptual models* [Documento académico].
https://archives.iw3c2.org/www2002/_west2001/iwwost01/files/contributions/DanielSchwabe/WWWCM99.pdf